Inference lab using the five-step template: Hypothesis → Visualise → Assumptions → Conduct → Conclude.

Learning objectives

- Decompose a seasonal time series into trend, seasonal, and residual components.
- Fit an ARIMA model automatically with `forecast::auto.arima`.
- Detect a mean shift using `changepoint::cpt.mean`.

Prerequisites

Regression; basic familiarity with the `ts` class.

Background

A time series differs from cross-sectional data in one crucial way: observations close in time are correlated with one another, so standard errors based on independence are wrong. Classical decomposition splits a series into a slowly varying trend, a periodic seasonal component, and a residual. ARIMA models capture the residual autocorrelation with autoregressive and moving-average terms, possibly after differencing to remove a unit root.

Change-point detection asks a different question: given a series, when (if ever) did the underlying mean or variance shift? The `changepoint` package implements several methods; `cpt.mean` with a penalised likelihood criterion is a common starting point. In epidemiology, change-points flag outbreaks, policy changes, and data-collection transitions.

A seasonally adjusted series is not the same as a detrended series. Seasonal adjustment removes only the regular period; detrending removes the slow evolution. Most anomalies you care about (an outbreak, an intervention) live in what remains.

Setup

```
library(tidyverse)
library(forecast)
library(changepoint)
set.seed(42)
theme_set(theme_minimal(base_size = 12))
```

1. Hypothesis

A simulated monthly series has a linear trend, a yearly seasonal cycle, and a mean shift at month 80. Decomposition, ARIMA, and change-point detection should each recover an interpretable piece.

2. Visualise

```
t  <- seq_len(months)
trend <- 0.08 * t
season <- 4 * sin(2 * pi * t / 12)
shift  <- if_else(t >= 80, 5, 0)
y      <- 20 + trend + season + shift + rnorm(months, 0, 1.5)
ts_y   <- ts(y, frequency = 12, start = c(2014, 1))

autoplot(ts_y) +
  labs(x = "Year", y = "Simulated rate")
```

3. Assumptions

STL decomposition assumes the seasonal period is fixed and known. ARIMA assumes a linear, time-invariant generating process after differencing. `cpt.mean` assumes the variance is approximately constant — change-points in variance would need `cpt.var`.

4. Conduct

```
autoplot(decomp)
```

```
fit_arima
```

```
autoplot(fc) +
  labs(x = "Year", y = "Simulated rate")
```

```
cpts(cpt)
```

```
tibble(t = t, y = as.numeric(ts_y)) |>
  ggplot(aes(t, y)) +
  geom_line() +
  geom_vline(xintercept = cpts(cpt), linetype = "dashed",
             colour = "firebrick") +
  labs(x = "Month index", y = "Rate")
```

5. Concluding statement

STL cleanly separated a linear trend, a 12-month seasonal cycle, and a residual containing the simulated mean shift. `auto.arima` selected `fit_arima$arma |> paste(collapse = ", ")` (p, q, P, Q, frequency, d, D). PELT detected change-points at months `paste(cpts(cpt), collapse = ", ")`, close to the simulated truth of 80.

Common pitfalls

- Using daily ARIMA on weekly-seasonal data without telling the model the period.
- Calling every bump a change-point; penalise harder and re-run.
- Using `lm()` on a time series as if residuals were independent.
- Forecasting far beyond the observed period without widening the intervals in the report.

Further reading

- Hyndman RJ, Athanasopoulos G. *Forecasting: Principles and Practice*.
- Killick R, Fearnhead P, Eckley IA (2012), *Optimal detection of changepoints with a linear computational cost*.
- Box GEP, Jenkins GM. *Time Series Analysis: Forecasting and Control*.

Session info

See also — chapter index

- Methods in this chapter
- Find your method (Chapter 0)